ORBIT · PYTHON · PAPER 10

第 10 套

60 题 · 完整题目与答案

单选 30 多选 15 填空 / 简答 15

本套分节

一、单项选择题（30） 二、不定项选择题（15） 三、填空题（5） 四、程序代码阅读题（5） 五、程序设计题（5）

答案与解析紧随每题；本文件用于复习核对，建议先独立作答。

一、单项选择题

01 • 单项选择

以下代码的输出是什么？

```
def build_list(val, lst=None):
    if lst is None:
        lst = []
    lst.append(val)
    return lst
a = build_list(1)
b = build_list(2)
print(len(a), len(b))
```

A. 1 1
B. 2 2
C. 1 2
D. 2 1

答案 A. 1 1

解析

02 • 单项选择

关于列表的extend()方法，以下说法正确的是：

```
lst = [1, 2]
lst.extend([3, 4])
```

A. 返回新列表
B. 修改原列表
C. 报错
D. 不做任何操作

答案 B. 修改原列表

解析

03 • 单项选择

以下代码的输出是什么？

```
original = [1, 2, 3]
copy1 = original
copy2 = original[:]
original.append(4)
print(len(copy1), len(copy2))
```

A. 3 3
B. 4 4
C. 4 3
D. 3 4

答案 C. 4 3

解析

04 • 单项选择

关于元组的+=操作，以下说法正确的是：

```
t = (1, 2)
t += (3,)
```

A. 修改原元组
B. 创建新元组
C. 报错
D. t保持不变

答案 B. 创建新元组

解析

05 • 单项选择

以下代码的输出是什么？

```
def modify_tuple(t):
    t = t + (4,)
    return t
original = (1, 2, 3)
result = modify_tuple(original)
print(len(original), len(result))
```

A. 3 3
B. 4 4
C. 3 4
D. 4 3

答案 C. 3 4

解析

06 • 单项选择

关于字典的pop()方法，以下说法正确的是：

A. 不修改原字典
B. 修改原字典并返回删除的值
C. 键不存在时返回None
D. 不能指定默认值

答案 B. 修改原字典并返回删除的值

解析

07 • 单项选择

以下代码的输出是什么？

```
d = {'a': 1, 'b': 2}
val = d.pop('c', 0)
print(val, len(d))
```

A. None 2
B. 0 2
C. KeyError
D. 0 3

答案 B. 0 2

解析

08 • 单项选择

关于集合的difference()和difference_update()，以下说法正确的是：

A. 两者都修改原集合
B. difference()返回新集合，difference_update()修改原集合
C. 两者都返回新集合
D. 完全相同

答案 B. difference()返回新集合，difference_update()修改原集合

解析

09 • 单项选择

以下代码的输出是什么？

```
s1 = {1, 2, 3, 4}
s2 = {3, 4, 5, 6}
s3 = s1.difference(s2)
print(len(s1), len(s3))
```

A. 2 2
B. 4 2
C. 2 4
D. 4 4

答案 B. 4 2

解析

10 • 单项选择

关于字符串的split()方法，以下说法正确的是：

A. 修改原字符串
B. 返回列表
C. 返回元组
D. 返回字符串

答案 B. 返回列表

解析

11 • 单项选择

以下代码的输出是什么？

```
s = "a,b,c"
parts = s.split(',')
s = s.upper()
print(parts[0])
```

A. 'A'
B. 'a'
C. 'a,b,c'
D. 报错

答案 B. 'a'

解析

12 • 单项选择

关于列表的clear()和赋值为空列表，以下说法正确的是：

```
lst1 = [1, 2, 3]
lst2 = lst1
# 方法1: lst1.clear()
# 方法2: lst1 = []
```

A. 两种方法效果相同

B. clear()会影响lst2，赋值不会

C. 赋值会影响lst2，clear()不会

D. 都不影响lst2

答案 B. clear()会影响lst2，赋值不会

解析

13 • 单项选择

以下代码的输出是什么？

```
lst1 = [1, 2, 3]
lst2 = lst1
lst1 = []
print(len(lst2))
```

A. 0

B. 3

C. None

D. 报错

答案 B. 3

解析

14 • 单项选择

关于frozenset，以下说法正确的是：

A. 可以修改元素

B. 不可变的集合

C. 不能包含元素

D. 与set完全相同

答案 B. 不可变的集合

解析

15 • 单项选择

以下代码的输出是什么？

```
fs = frozenset([1, 2, 3])
s = {fs}
print(len(s))
```

A. 0

B. 1
C. 3
D. 报错

答案 B. 1

解析

16 • 单项选择

Pandas中，以下哪种操作最安全（不会产生警告）？

A. df[df['A'] > 0]['B'] = 1
B. df.loc[df['A'] > 0, 'B'] = 1
C. df['B'][df['A'] > 0] = 1
D. 都一样

答案 B. df.loc[df['A'] > 0, 'B'] = 1

解析

17 • 单项选择

以下代码的输出是什么？

```
import pandas as pd
df = pd.DataFrame({'A': [1, 2, 3]})
df_copy = df.copy(deep=True)
df['A'] = [10, 20, 30]
print(df_copy['A'].iloc[0])
```

A. 1
B. 10
C. None
D. 报错

答案 A. 1

解析

18 • 单项选择

关于列表的+和+=操作符，以下说法正确的是：

A. 两者都修改原列表
B. +创建新列表，+=修改原列表
C. +修改原列表，+=创建新列表
D. 两者都创建新列表

答案 B. +创建新列表，+=修改原列表

解析

19 • 单项选择

以下代码的输出是什么？

```
a = [1, 2]
b = a
a = a + [3]
print(id(a) == id(b))
```

A. True
B. False
C. 可能是True或False
D. 报错

答案 B. False

解析

20 • 单项选择

关于字典的get()方法的默认值，以下说法正确的是：

A. 默认值是None
B. 默认值是0
C. 默认值是空字符串
D. 必须指定默认值

答案 A. 默认值是None

解析

21 • 单项选择

以下代码的输出是什么？

```
d = {'a': 1}
val = d.get('b')
print(val)
```

A. 0
B. None
C. ''
D. KeyError

答案 B. None

解析

22 • 单项选择

关于元组的不可变性和列表的可变性，以下说法正确的是：

A. 元组总是比列表占用更少内存
B. 元组的访问速度总是更快
C. 元组可以作为字典键，列表不能
D. 以上都正确

答案 C. 元组可以作为字典键，列表不能

解析

23 • 单项选择

以下代码会发生什么？

```
d = {[1, 2]: 'value'}
```

A. 正常执行
B. 抛出TypeError
C. 列表被转换为元组
D. 创建空字典

答案 B. 抛出TypeError

解析

24 • 单项选择

关于集合的|、&、-运算符，以下说法正确的是：

A. 都修改原集合
B. 都返回新集合
C. 只有|修改原集合
D. 只有&修改原集合

答案 B. 都返回新集合

解析

25 • 单项选择

以下代码的输出是什么？

```
s1 = {1, 2, 3}
s2 = {2, 3, 4}
s3 = s1 | s2
print(len(s1), len(s3))
```

A. 3 4
B. 4 4
C. 3 3
D. 4 3

答案 A. 3 4

解析

26 • 单项选择

关于字符串的format()方法，以下说法正确的是：

A. 修改原字符串
B. 返回新字符串
C. 只能格式化数字
D. 不能使用

答案 B. 返回新字符串

解析

27 • 单项选择

以下代码的输出是什么？

```
template = "Hello, {}"
result = template.format("World")
print(template)
```

A. "Hello, World"
B. "Hello, {}"
C. "World"
D. 报错

答案 B. "Hello, {}"

解析

28 • 单项选择

关于列表的sort()方法的reverse参数，以下说法正确的是：

A. reverse=True降序排序
B. reverse=False降序排序
C. 默认是True
D. 不能使用

答案 A. reverse=True降序排序

解析

29 • 单项选择

以下代码的输出是什么？

```
lst = [3, 1, 4, 1, 5]
lst.sort(reverse=True)
print(lst[0])
```

A. 1
B. 3
C. 5
D. 报错

答案 C. 5

解析

30 • 单项选择

关于Pandas的copy()方法的deep参数，以下说法正确的是：

A. deep=True创建深拷贝
B. deep=False创建浅拷贝
C. 默认是deep=True
D. 以上都正确

答案 D. 以上都正确

解析

二、不定项选择题

31 • 不定项选择

以下哪些操作会创建对象的副本？

A. list.copy()
B. dict.copy()
C. set.copy()
D. str.copy()

答案 A. list.copy()；B. dict.copy()；C. set.copy()

解析

32 • 不定项选择

关于可变对象作为类属性，以下说法正确的有：

A. 所有实例共享同一对象
B. 可能导致意外行为
C. 应该在__init__中初始化
D. 没有问题

答案 A. 所有实例共享同一对象；B. 可能导致意外行为；C. 应该在__init__中初始化

解析

33 • 不定项选择

以下哪些类型的对象可以作为集合的元素？

A. int
B. str
C. tuple（包含不可变元素）
D. list

答案 A. int；B. str；C. tuple（包含不可变元素）

解析

34 • 不定项选择

关于列表的切片操作，以下说法正确的有：

A. 创建新列表
B. 是浅拷贝
C. 可以用于删除元素
D. 可以用于插入元素

答案 A. 创建新列表；B. 是浅拷贝；C. 可以用于删除元素；D. 可以用于插入元素

解析

35 • 不定项选择

以下哪些方法会修改原字典？

A. update()
B. pop()
C. popitem()
D. get()

答案 A. update()；B. pop()；C. popitem()

解析

36 • 不定项选择

关于元组包含列表的情况，以下说法正确的有：

A. 元组本身不可变
B. 列表内容可以修改
C. 这样的元组不可哈希
D. 不能创建这样的元组

答案 A. 元组本身不可变；B. 列表内容可以修改；C. 这样的元组不可哈希

解析

37 • 不定项选择

以下哪些是字符串的不可变性带来的影响？

A. 字符串拼接效率较低
B. 字符串可以作为字典键
C. 字符串方法都返回新字符串
D. 字符串占用更多内存

答案 A. 字符串拼接效率较低；B. 字符串可以作为字典键；C. 字符串方法都返回新字符串

解析

38 • 不定项选择

关于深拷贝和浅拷贝的选择，以下说法正确的有：

A. 简单对象用浅拷贝
B. 嵌套对象需要完全独立时用深拷贝
C. 深拷贝更慢
D. 总是应该用深拷贝

答案 A. 简单对象用浅拷贝；B. 嵌套对象需要完全独立时用深拷贝；C. 深拷贝更慢

解析

39 • 不定项选择

以下哪些操作不会修改原列表？

A. `sorted(lst)`
B. `reversed(lst)`
C. `lst[:]`
D. `list(lst)`

答案 A. `sorted(lst)`；B. `reversed(lst)`；C. `lst[:]`；D. `list(lst)`

解析

40 • 不定项选择

关于Pandas的视图和副本，以下说法正确的有：

A. 视图修改会影响原数据
B. 副本修改不影响原数据
C. 使用`copy()`确保获得副本
D. 链式索引可能产生副本

答案 A. 视图修改会影响原数据；B. 副本修改不影响原数据；C. 使用`copy()`确保获得副本；D. 链式索引可能产生副本

解析

41 • 不定项选择

以下哪些是不可变类型？

A. `int`
B. `float`
C. `str`

D. bool

答案 A. int；B. float；C. str；D. bool

解析

42 • 不定项选择

关于函数参数的可变性问题，以下说法正确的有：

A. 传递可变对象时要小心
B. 可以在函数内创建副本
C. 使用None作为默认值更安全
D. 不可变对象没有这个问题

答案 A. 传递可变对象时要小心；B. 可以在函数内创建副本；C. 使用None作为默认值更安全；D. 不可变对象没有这个问题

解析

43 • 不定项选择

以下哪些集合操作会修改原集合？

A. add()
B. union()
C. update()
D. intersection()

答案 A. add()；C. update()

解析

44 • 不定项选择

关于列表推导式和可变性，以下说法正确的有：

A. 创建新列表
B. 不修改原数据
C. 可以包含可变对象
D. 总是比循环安全

答案 A. 创建新列表；B. 不修改原数据；C. 可以包含可变对象

解析

45 • 不定项选择

以下哪些是避免可变默认参数问题的方法？

A. 使用None作为默认值
B. 在函数内创建新对象
C. 使用不可变对象
D. 使用global变量

答案 A. 使用None作为默认值；B. 在函数内创建新对象；C. 使用不可变对象

解析

三、填空题

46 • 填空 / 简答

可变对象作为默认参数的问题可以通过使用 _____ 作为默认值，然后在函数内部判断并创建 _____ 来解决。

答案 None ， 新对象

解析 新对象

就地修改方法vs返回新对象的方法：

- 就地修改方法（如 list.sort() 、 list.reverse() ）返回 None
- 返回新对象的方法（如 sorted() 、 reversed() ）返回新对象，原对象不变

47 • 填空 / 简答

列表的 _____ 方法会就地修改列表，而 _____ 函数返回排序后的新列表。

答案 sort() ， sorted()

解析 列表排序的两种方式（与第7、9套相同考点）

48 • 填空 / 简答

元组的不可变性是指 _____ 不可变，但元组中包含的 _____ 对象的内容可以被修改。

答案 元素引用

解析 元素引用 （或
（或 元组本身
元组本身 ），
）， 可变
可变

元组的不可变性是浅层的：

- 元组本身不可变（不能修改元组结构）
- 但元组内的可变对象可以被修改

49 • 填空 / 简答

在Pandas中，使用 _____ 索引可能产生SettingWithCopyWarning，应该使用 _____ 方法来避免。

答案 链式

解析 链式 ， copy()

Pandas避免链式索引警告的方法

50 • 填空 / 简答

字典的 _____ 方法会删除并返回键值对，而 _____ 方法只是查询不修改字典。

答案 pop() （或

解析 （或 popitem() ），
）， get()

字典的修改方法vs查询方法

四、程序代码阅读题

51 • 填空 / 简答

阅读以下代码，回答问题：

```
class TaskQueue:
    tasks = []
    def add_task(self, task):
        self.tasks.append(task)
    def get_tasks(self):
        return self.tasks
    def clear_tasks(self):
        self.tasks = []
q1 = TaskQueue()
q2 = TaskQueue()
q1.add_task("Task 1")
q2.add_task("Task 2")
q1.clear_tasks()
print(len(q1.get_tasks()))
print(len(q2.get_tasks()))
```

问题：

- (1) 两个print的输出分别是什么？
- (2) 为什么clear_tasks()的行为不符合预期？
- (3) 如何正确实现这个类？

答案 类问题

- (1) 输出：0, 1
- (2) clear_tasks() 中的 self.tasks = [] 创建了新的实例属性，而不是清空类属性
- (3) 应该在 __init__ 中初始化实例属性：

```
def __init__(self):
    self.tasks = []
```

解析

52 • 填空 / 简答

阅读以下代码，回答问题：

```
import copy
class Node:
    def __init__(self, value, children=None):
        self.value = value
        self.children = children if children else []
    def add_child(self, child):
        self.children.append(child)
root = Node(1)
child1 = Node(2)
child2 = Node(3)
root.add_child(child1)
root.add_child(child2)
# 浅拷贝
root_shallow = copy.copy(root)
root_shallow.value = 10
```

```
# 深拷贝
root_deep = copy.deepcopy(root)
root_deep.children[0].value = 20
print(root.value)
print(root.children[0].value)
print(root_shallow.children[0].value)
```

问题：

- (1) 三个print的输出分别是什么？
- (2) 浅拷贝和深拷贝的区别在这里如何体现？
- (3) 什么时候需要使用深拷贝？

答案 浅拷贝vs深拷贝
深拷贝

- (1) 输出：1, 20, 20
- (2) 浅拷贝只复制对象本身，children列表仍是共享的；深拷贝递归复制所有对象
- (3) 当对象包含嵌套的可变对象，且需要完全独立的副本时

解析

53 • 填空 / 简答

阅读以下代码，回答问题：

```
import pandas as pd
df = pd.DataFrame({
    'group': ['A', 'A', 'B', 'B'],
    'value': [1, 2, 3, 4]
})
# 方法1
grouped = df.groupby('group')
for name, group in grouped:
    group['new_col'] = group['value'] * 2
# 方法2
df['new_col'] = df.groupby('group')['value'].transform(lambda x: x * 2)
print('new_col' in df.columns)
```

问题：

- (1) 输出是什么？
- (2) 为什么方法1不会修改原DataFrame？
- (3) transform()方法的作用是什么？

答案 分组操作

- (1) 输出：True
- (2) 方法1中的group是副本，修改不影响原DataFrame
- (3) transform()对每个分组应用函数并保持原DataFrame的形状

解析

54 • 填空 / 简答

阅读以下代码，回答问题：

```
def process_config(config, defaults={'timeout': 30, 'retries': 3}):
    defaults.update(config)
```

```
    return defaults
config1 = process_config({'timeout': 60})
config2 = process_config({'retries': 5})
print(config1)
print(config2)
print(config1 is config2)
```

问题：

- (1) 三个print的输出分别是什么？
- (2) 这个函数有什么严重问题？
- (3) 如何修复这个函数？

答案 • (1) 输出：

```
{'timeout': 60, 'retries': 5}
True
```

- (2) 默认参数defaults是可变字典，被所有调用共享并累积修改
- (3) 修复方法：

```
def process_config(config, defaults=None):
    if defaults is None:
        defaults = {'timeout': 30, 'retries': 3}
    result = defaults.copy()
    result.update(config)
    return result
```

解析

55 • 填空 / 简答

阅读以下代码，回答问题：

```
class ImmutablePoint:
    __slots__ = ('x', 'y')
    def __init__(self, x, y):
        object.__setattr__(self, 'x', x)
        object.__setattr__(self, 'y', y)
    def __setattr__(self, name, value):
        raise AttributeError("Cannot modify immutable object")
    def __hash__(self):
        return hash((self.x, self.y))
p1 = ImmutablePoint(1, 2)
p2 = ImmutablePoint(1, 2)
print(p1 == p2)
print(hash(p1) == hash(p2))
d = {p1: 'point1'}
d[p2] = 'point2'
print(len(d))
```

问题：

- (1) 三个print的输出分别是什么？
- (2) 为什么需要实现__hash__()？
- (3) 这个类还缺少什么方法？

答案 不可变Point类

- (1) 输出：False, True, 1
- (2) 实现 __hash__() 使对象可以作为字典键和集合元素
- (3) 缺少 __eq__() 方法，导致p1 == p2返回False

解析

五、程序设计题

56 • 填空 / 简答

实现一个写时复制（Copy-on-Write）的列表容器：

1. COWList类：

- 初始时不复制数据
- 第一次修改时才复制
- 多个实例可以共享数据

2. 功能要求

功能要求：

- 支持所有列表操作
- 追踪引用计数
- 优化内存使用

3. 性能测试

性能测试：

- 对比普通列表
- 测量内存占用
- 测量操作速度

答案

```
import copy
from weakref import WeakValueDictionary
class COWList:
    """
    写时复制（Copy-on-Write）列表容器
    设计思路：
    1. 多个实例共享同一份数据
    2. 只有在修改时才复制数据
    3. 使用引用计数追踪共享
    4. 优化内存使用
    特性：
    - 读操作：O(1)，无复制开销
    - 写操作：首次写时复制，后续O(1)
    - 内存优化：多个实例共享数据
    使用示例：
    list1 = COWList([1, 2, 3])
    list2 = COWList.from_cow(list1) # 共享数据
    list2.append(4) # 触发复制
    """
    # 类级别的数据存储池
    _data_pool = WeakValueDictionary()
    _next_id = 0
```

```
    def __init__(self, data=None):
        """
        初始化COW列表
        参数：
        - data: 初始数据
        """
        if data is None:
            data = []
    # 为数据分配唯一ID
    self._data_id = COWList._next_id
    COWList._next_id += 1
    # 存储数据
    self._data = list(data)
    self._is_shared = False
    self._ref_count = 1
@classmethod
def from_cow(cls, other):
    """
    从另一个COW列表创建共享实例
    参数：
    - other: 另一个COWList实例
    返回：
    - COWList: 新实例，共享数据
    """
    new_instance = cls.__new__(cls)
    new_instance._data = other._data
    new_instance._data_id = other._data_id
    new_instance._is_shared = True
    other._is_shared = True
    new_instance._ref_count = other._ref_count
    other._ref_count += 1
    return new_instance
def _ensure_unique(self):
    """确保数据独占（写时复制）"""
    if self._is_shared:
        print(f"[COW] 触发复制，原引用计数: {self._ref_count}")
        self._data = self._data.copy()
        self._is_shared = False
        self._ref_count = 1
# === 列表操作 ===
def append(self, item):
    """添加元素"""
    self._ensure_unique()
    self._data.append(item)
def extend(self, items):
    """扩展列表"""
    self._ensure_unique()
    self._data.extend(items)
```

```
def insert(self, index, item):
    """插入元素"""
    self._ensure_unique()
    self._data.insert(index, item)
def remove(self, item):
    """移除元素"""
    self._ensure_unique()
    self._data.remove(item)
def pop(self, index=-1):
    """弹出元素"""
    self._ensure_unique()
    return self._data.pop(index)
def clear(self):
    """清空列表"""
    self._ensure_unique()
    self._data.clear()
def sort(self, **kwargs):
    """排序"""
    self._ensure_unique()
    self._data.sort(**kwargs)
# === 只读操作（不触发复制）===
def __getitem__(self, index):
    """获取元素"""
    return self._data[index]
def __len__(self):
    """长度"""
        return len(self._data)
    def __iter__(self):
        """迭代"""
        return iter(self._data)
    def __repr__(self):
        """字符串表示"""
        shared_marker = " [共享]" if self._is_shared else ""
        return f"COWList({self._data}){shared_marker}"
    def get_memory_info(self):
        """获取内存信息"""
        import sys
        return {
            'data_size': sys.getsizeof(self._data),
            'is_shared': self._is_shared,
            'ref_count': self._ref_count,
            'data_id': id(self._data)
        }
# === 使用示例 ===
print("=== 写时复制容器示例 ===\n")
# 示例1：共享数据
print("示例1：数据共享")
list1 = COWList([1, 2, 3, 4, 5])
```

```
print(f"list1: {list1}")
print(f"内存信息: {list1.get_memory_info()}")
list2 = COWList.from_cow(list1)
print(f"\nlist2（共享自list1）: {list2}")
print(f"list1数据ID: {id(list1._data)}")
print(f"list2数据ID: {id(list2._data)}")
print(f"数据共享: {id(list1._data) == id(list2._data)}")
# 示例2：写时复制
print("\n示例2：写时复制触发")
print("修改list2...")
list2.append(6)
print(f"list1: {list1}")
print(f"list2: {list2}")
print(f"list1数据ID: {id(list1._data)}")
print(f"list2数据ID: {id(list2._data)}")
print(f"数据已分离: {id(list1._data) != id(list2._data)}")
# 示例3：性能对比
print("\n示例3：性能测试")
import time
import sys
def test_normal_copy(n):
    """测试普通复制"""
    lists = []
    start = time.time()
    base = list(range(n))
    for _ in range(100):
        lists.append(base.copy())
    return time.time() - start, sum(sys.getsizeof(lst) for lst in lists)
def test_cow(n):
    """测试COW"""
    lists = []
    start = time.time()
    base = COWList(list(range(n)))
    for _ in range(100):
        lists.append(COWList.from_cow(base))
    return time.time() - start, sum(sys.getsizeof(lst._data) for lst in lists)
n = 1000
normal_time, normal_memory = test_normal_copy(n)
cow_time, cow_memory = test_cow(n)
print(f"数据大小: {n}个元素")
print(f"普通复制: 时间={normal_time:.4f}s, 内存≈{normal_memory}字节")
print(f"COW复制: 时间={cow_time:.4f}s, 内存≈{cow_memory}字节")
print(f"内存节省: {(1 - cow_memory/normal_memory)*100:.1f}%")
```

解析

57 • 填空 / 简答

实现一个完整的不可变配置系统：
1. Config类：
• 完全不可变
• 支持嵌套配置
• 提供便捷访问
2. ConfigBuilder：
• 构建器模式
• 验证配置
• 生成不可变实例
3. 功能
功能：
• 配置合并
• 配置继承
• 配置导出

答案
```
from collections.abc import Mapping
import json
from pathlib import Path
class Config(Mapping):
    """
    完全不可变的配置对象
    特性：
    1. 初始化后完全不可修改
    2. 支持嵌套配置（点号访问）
    3. 可哈希，能作为字典键
    4. 支持配置合并和继承
    使用示例：
    config = Config({'database': {'host': 'localhost'}})
    print(config['database.host'])
    """
    __slots__ = ('_data', '_hash')
    def __init__(self, data):
        """
        初始化配置
        参数：
        - data: 配置字典
        """
        # 递归转换为不可变
        object.__setattr__(self, '_data', self._freeze(data))
        object.__setattr__(self, '_hash', None)
    def _freeze(self, obj):
        """递归冻结对象"""
        if isinstance(obj, dict):
            return {k: self._freeze(v) for k, v in obj.items()}
        elif isinstance(obj, list):
            return tuple(self._freeze(item) for item in obj)
    elif isinstance(obj, set):
        return frozenset(self._freeze(item) for item in obj)
```

```
    else:
        return obj
def get(self, path, default=None):
    """
    使用点号路径获取配置
    参数：
    - path: 配置路径，如 'database.host'
    - default: 默认值
    返回：
    - 配置值或默认值
    """
    keys = path.split('.')
    value = self._data
    for key in keys:
        if isinstance(value, dict) and key in value:
            value = value[key]
        else:
            return default
    # 如果是字典，包装为Config
    if isinstance(value, dict):
        return Config(value)
    return value
# === Mapping接口 ===
def __getitem__(self, key):
    """支持字典访问"""
    if '.' in key:
        return self.get(key)
    value = self._data[key]
    if isinstance(value, dict):
        return Config(value)
    return value
def __iter__(self):
    """迭代键"""
    return iter(self._data)
def __len__(self):
    """长度"""
    return len(self._data)
# === 禁止修改 ===
def __setattr__(self, name, value):
    """禁止设置属性"""
    raise AttributeError("Config对象不可变")
def __setitem__(self, key, value):
    """禁止设置项"""
    raise TypeError("Config对象不可变")
# === 哈希支持 ===
def __hash__(self):
    """计算哈希值"""
    if self._hash is None:
```

```
        object.__setattr__(self, '_hash', hash(self._to_hashable(self._data)))
    return self._hash
def _to_hashable(self, obj):
    """转换为可哈希对象"""
    if isinstance(obj, dict):
        return tuple(sorted((k, self._to_hashable(v)) for k, v in obj.items()))
    elif isinstance(obj, (list, tuple)):
        return tuple(self._to_hashable(item) for item in obj)
    elif isinstance(obj, set):
        return frozenset(self._to_hashable(item) for item in obj)
    else:
        return obj
def __repr__(self):
    """字符串表示"""
    return f"Config({self._data})"
def merge(self, other):
    """
    合并配置，返回新Config
    参数：
    - other: 另一个配置字典或Config对象
    返回：
    - Config: 合并后的新配置
    """
    if isinstance(other, Config):
        other = other._data
    merged = self._deep_merge(self._data, other)
    return Config(merged)
def _deep_merge(self, base, update):
    """深度合并字典"""
    result = base.copy()
    for key, value in update.items():
        if key in result and isinstance(result[key], dict) and isinstance(value, dict)
            result[key] = self._deep_merge(result[key], value)
        else:
            result[key] = value
    return result
def to_dict(self):
    """转换为普通字典"""
    return self._unfreeze(self._data)
def _unfreeze(self, obj):
    """递归解冻对象"""
    if isinstance(obj, dict):
        return {k: self._unfreeze(v) for k, v in obj.items()}
    elif isinstance(obj, tuple):
        return [self._unfreeze(item) for item in obj]
    elif isinstance(obj, frozenset):
        return {self._unfreeze(item) for item in obj}
    else:
```

```
        return obj
class ConfigBuilder:
    """
    配置构建器
    功能：
    1. 流式API构建配置
    2. 配置验证
    3. 从文件加载
    4. 配置继承
    使用示例：
    config = (ConfigBuilder()
        .set('app.name', 'MyApp')
        .set('app.version', '1.0')
        .validate()
        .build())
    """
    def __init__(self):
        """初始化构建器"""
        self._config = {}
        self._validators = []
    def set(self, path, value):
        """
        设置配置值
        参数：
        - path: 配置路径（支持点号）
        - value: 配置值
        返回：
        - self: 支持链式调用
        """
        keys = path.split('.')
        current = self._config
        for key in keys[:-1]:
        if key not in current:
            current[key] = {}
        current = current[key]
    current[keys[-1]] = value
    return self
def merge(self, config_dict):
    """
    合并配置字典
    参数：
    - config_dict: 要合并的配置
    返回：
    - self: 支持链式调用
    """
    self._deep_merge(self._config, config_dict)
    return self
def _deep_merge(self, base, update):
```

```
    """深度合并"""
    for key, value in update.items():
        if key in base and isinstance(base[key], dict) and isinstance(value, dict):
            self._deep_merge(base[key], value)
        else:
            base[key] = value
def load_from_file(self, filepath):
    """
    从JSON文件加载配置
    参数:
    - filepath: 文件路径
    返回:
    - self: 支持链式调用
    """
    path = Path(filepath)
    if path.exists():
        with path.open('r', encoding='utf-8') as f:
            config = json.load(f)
        self.merge(config)
    return self
def add_validator(self, validator_func, error_msg=None):
    """
    添加验证器
    参数:
    - validator_func: 验证函数，接收config字典，返回bool
    - error_msg: 错误消息
    返回:
    - self: 支持链式调用
    """
    self._validators.append((validator_func, error_msg))
    return self
def validate(self):
    """
    验证配置
    返回:
    - self: 支持链式调用
    抛出:
    - ValueError: 验证失败
    """
    for validator, error_msg in self._validators:
        if not validator(self._config):
            msg = error_msg or f"验证失败: {validator.__name__}"
            raise ValueError(msg)
    return self
def build(self):
    """
    构建不可变配置对象
    返回:
```

```
        - Config: 不可变配置
        """
        return Config(self._config)
    def save_to_file(self, filepath):
        """
        保存配置到文件
        参数:
        - filepath: 文件路径
        """
        path = Path(filepath)
        with path.open('w', encoding='utf-8') as f:
            json.dump(self._config, f, indent=2, ensure_ascii=False)
# === 使用示例 ===
print("=== 不可变配置系统示例 ===\n")
# 示例1：创建配置
print("示例1：创建和访问配置")
config = Config({
    'app': {
        'name': 'MyApp',
        'version': '1.0.0',
        'debug': False
    },
    'database': {
        'host': 'localhost',
        'port': 5432,
        'credentials': {
            'user': 'admin',
            'password': 'secret'
    },
    'features': ['auth', 'api', 'cache']
})
print(f"应用名称: {config['app']['name']}")
print(f"数据库主机: {config.get('database.host')}")
print(f"用户名: {config.get('database.credentials.user')}")
print(f"功能列表: {config['features']}")
# 示例2：配置合并
print("\n示例2：配置合并")
base_config = Config({'app': {'name': 'Base', 'debug': True}})
override_config = {'app': {'debug': False}, 'new_key': 'value'}
merged = base_config.merge(override_config)
print(f"基础配置: {base_config}")
print(f"合并后: {merged}")
# 示例3：使用构建器
print("\n示例3：使用ConfigBuilder")
def validate_port(config):
    """验证端口范围"""
    if 'database' in config and 'port' in config['database']:
        port = config['database']['port']
```

```
        return 1024 <= port <= 65535
    return True
def validate_required(config):
    """验证必需字段"""
    return 'app' in config and 'name' in config['app']
built_config = (ConfigBuilder()
    .set('app.name', 'BuilderApp')
    .set('app.version', '2.0.0')
    .set('database.host', 'localhost')
    .set('database.port', 5432)
    .add_validator(validate_port, "端口必须在1024-65535之间")
    .add_validator(validate_required, "缺少必需的app.name字段")
    .validate()
    .build())
print(f"构建的配置: {built_config}")
print(f"配置哈希: {hash(built_config)}")
# 示例4：配置作为字典键
print("\n示例4：配置作为缓存键")
cache = {
    built_config: "cached_data_1",
    merged: "cached_data_2"
}
print(f"缓存条目数: {len(cache)}")
```

解析

58 • 填空 / 简答

实现完整的函数参数验证框架：

1. 验证装饰器

验证装饰器：

- @validate_types ：类型验证
- @validate_range ：范围验证
- @validate_immutable ：不可变验证

2. 自动保护

自动保护：

- 自动复制可变参数
- 防止意外修改
- 详细错误信息

3. 性能优化

性能优化：

- 缓存验证结果
- 最小化复制
- 可配置验证级别

答案

```
from functools import wraps
import inspect
from typing import get_type_hints, Any
import copy
def validate_types(func):
```

```
    """
    类型验证装饰器
    功能：
    - 基于类型注解验证参数类型
    - 类型不匹配时抛出TypeError
    - 支持可选类型（Union, Optional）
    使用示例：
    @validate_types
    def func(name: str, age: int) -> str:
        return f"{name} is {age}"
    """
    @wraps(func)
    def wrapper(*args, **kwargs):
        # 获取类型提示
        hints = get_type_hints(func)
        sig = inspect.signature(func)
        bound = sig.bind(*args, **kwargs)
        bound.apply_defaults()
        # 验证每个参数
        for param_name, param_value in bound.arguments.items():
            if param_name in hints and param_name != 'return':
                expected_type = hints[param_name]
                # 处理泛型类型
                if hasattr(expected_type, '__origin__'):
                # Union, Optional等
                continue
                if not isinstance(param_value, expected_type):
                raise TypeError(
                f"参数'{param_name}'类型错误: "
                f"期望{expected_type.__name__}, "
                f"得到{type(param_value).__name__}"
                )
        return func(*args, **kwargs)
    return wrapper
def validate_range(**ranges):
    """
    范围验证装饰器
    功能：
    - 验证数值参数的范围
    - 支持min和max限制
    使用示例：
    @validate_range(age=(0, 150), score=(0, 100))
    def func(age, score):
        ...
    """
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
```

```
            sig = inspect.signature(func)
            bound = sig.bind(*args, **kwargs)
            bound.apply_defaults()
            for param_name, (min_val, max_val) in ranges.items():
                if param_name in bound.arguments:
                value = bound.arguments[param_name]
                if not (min_val <= value <= max_val):
                raise ValueError(
                f"参数'{param_name}'超出范围: "
                f"{value} 不在 [{min_val}, {max_val}] 内"
                )
            return func(*args, **kwargs)
        return wrapper
    return decorator
def validate_immutable(*param_names):
    """
    不可变验证装饰器
    功能:
    - 确保指定参数是不可变对象
    - 可变对象会抛出TypeError
    使用示例:
    @validate_immutable('config', 'settings')
    def func(config, settings):
        ...
    """
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            sig = inspect.signature(func)
            bound = sig.bind(*args, **kwargs)
            immutable_types = (int, float, str, tuple, frozenset, bytes, bool, type(None))
            for param_name in param_names:
                if param_name in bound.arguments:
                value = bound.arguments[param_name]
                if not isinstance(value, immutable_types):
                # 检查元组是否包含可变对象
                if isinstance(value, tuple):
                if any(not isinstance(item, immutable_types) for item in value
                raise TypeError(
                f"参数'{param_name}'必须是不可变对象, "
                f"但元组包含可变元素"
                )
                else:
                raise TypeError(
                f"参数'{param_name}'必须是不可变对象, "
                f"得到{type(value).__name__}"
                )
            return func(*args, **kwargs)
```

```
        return wrapper
    return decorator
def auto_copy_params(deep=False):
    """
    自动复制可变参数装饰器
    功能：
    - 自动检测并复制可变参数
    - 防止函数修改外部对象
    - 可配置深拷贝或浅拷贝
    使用示例：
    @auto_copy_params(deep=True)
    def func(data):
        data.append(1) # 不会影响外部
    """
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            def is_mutable(obj):
                immutable = (int, float, str, tuple, frozenset, bytes, bool, type(None))
                return not isinstance(obj, immutable)
            # 复制可变参数
            new_args = []
            for arg in args:
                if is_mutable(arg):
                new_args.append(copy.deepcopy(arg) if deep else copy.copy(arg))
                else:
                new_args.append(arg)
            new_kwargs = {}
            for key, value in kwargs.items():
                if is_mutable(value):
                new_kwargs[key] = copy.deepcopy(value) if deep else copy.copy(value)
                else:
                new_kwargs[key] = value
            return func(*new_args, **new_kwargs)
        return wrapper
    return decorator
# === 使用示例 ===
print("=== 参数验证框架示例 ===\n")
# 示例1：类型验证
print("示例1：类型验证")
@validate_types
def greet(name: str, age: int, active: bool = True) -> str:
    return f"{name}, {age}岁, 活跃:{active}"
try:
    print(greet("Alice", 30))
    print(greet("Bob", "25"))
# 类型错误
except TypeError as e:
```

```
    print(f"  {e}")
# 示例2：范围验证
print("\n示例2：范围验证")
@validate_range(age=(0, 150), score=(0, 100))
def register_student(name, age, score):
    return f"{name}: {age}岁, 分数{score}"
try:
    print(register_student("Alice", 20, 95))
    print(register_student("Bob", 200, 95))
# 年龄超出范围
except ValueError as e:
    print(f"  {e}")
# 示例3：不可变性验证
print("\n示例3：不可变性验证")
@validate_immutable('config')
def process_config(config):
    return f"处理配置: {config}"
try:
    print(process_config((1, 2, 3)))
    print(process_config([1, 2, 3]))
# 可变对象
except TypeError as e:
    print(f"  {e}")
# 示例4：自动复制参数
print("\n示例4：自动参数保护")
@auto_copy_params(deep=True)
def modify_data(data):
    data.append(999)
    return data
original = [1, 2, 3]
result = modify_data(original)
print(f"原数据: {original}")
print(f"返回值: {result}")
# 示例5：组合使用
print("\n示例5：组合多个验证器")
@validate_types
@validate_range(age=(0, 150))
@auto_copy_params(deep=True)
def update_user(name: str, age: int, hobbies: list):
    hobbies.append("reading")
    return {'name': name, 'age': age, 'hobbies': hobbies}
user_hobbies = ['coding', 'music']
result = update_user("Alice", 25, user_hobbies)
print(f"原爱好列表: {user_hobbies}")
print(f"更新结果: {result}")
```

评分说明：

- 单选题：每题1分，共30分
- 多选题：每题2分，选对但不全得1分，有错选不得分，共30分

- 填空题：每空1分，共10分
- 代码阅读题：每题3分，共15分
- 程序设计题：5选3，每题5分，共15分
- 总分：

总分：100分

解析

59 • 填空 / 简答

实现数据对象的完整版本管理：

1. VersionedObject类：

- 自动版本控制
- 修改历史记录
- 版本回滚

2. 差异追踪

差异追踪：

- 记录增量变化
- 最小化存储
- 快速恢复

3. 分支管理

分支管理：

- 创建分支
- 合并分支
- 冲突解决

答案 （原卷为 5 选 3，未提供本题参考答案；考试时任选 3 题作答即可）

解析

60 • 填空 / 简答

实现Pandas的安全操作封装：

答案 （原卷为 5 选 3，未提供本题参考答案；考试时任选 3 题作答即可）

解析